



**DOCKER**



# Introduction aux Conteneurs et Exploration de Docker

- Comprendre les fondamentaux des conteneurs
- Exploration de Docker : Introduction à l'écosystème Docker, ses composants et son utilisation
- Discussions et exemples pratiques pour illustrer les concepts

---

# Comprendre les fondamentaux des conteneurs



# Contexte

- Des cycles de développement plus courts
- Réduction des échecs de déploiement, des rollback et du temps de récupération
- Le style architectural des micro-services devient une norme pour la construction de systèmes déployés en continu
- Un coup de pouce pour votre carrière
- Les outils utilisés dans l'environnement DevOps (comme Docker, Jenkins, Puppet...) sont courants et largement adoptés dans le développement de solutions évolutives.
- Réduction des coûts
- Scalabilité
- Des cycles d'innovation plus rapides, où les applications sont prêtes à être utilisées beaucoup plus rapidement

---

# Exploration de Docker : Introduction à l'écosystème Docker, ses composants et son utilisation



# Virtualisation

- Docker -> Virtualisation : Machines virtuelles
- Concepte qui existe depuis les années 60 (Mainframe IBM 360)
- Emuler un ordinateur physique à l'aide d'un logiciel appelé Hyperviseur
- Docker est une solution de virtualisation différente d'une machine virtuelle



# Avant la virtualisation

Avant la virtualisation, utilisation d'une ou de plusieurs machines physiques en fonction du nombre d'applications à faire tourner

Un ordinateur hébergeait un OS, des librairies et des commandes, une ou plusieurs applications

Une telle architecture présente des inconvénients majeurs :

- des coûts d'investissement énorme : installer l'OS, installer les appli, installer les librairies etc
- des coûts de maintenances élevées et des interruptions de services longues et coûteuses en cas de pannes, de problèmes, ou de migration
- Des solutions de plus en plus moderne



## La machine Virtuelle

- L'ordinateur qui accueille la machine virtuelle s'appelle Hôte
- Dans une Hôte on trouve :
  - un OS
  - Des librairies
  - Des commandes
  - un hyperviseur
- Les VM, puisqu'elles s'apparentent à des machines physiques comportent :
  - son OS
  - Ses librairies
  - Ses commandes
  - Ses applications



## Avantages

- Réduction des coûts matériels
- Facilité d'installation
- Rapidité d'installation

## Inconvénients

- Son OS
- Ses librairies
- Ses commandes
- Ressources dédiées, non utilisable par d'autres VM (CPU, Mémoire et Disque)



## Hyperviseur avec un OS sur l'Hôte

- VMWare (VMWare)
- Virtualbox (Oracle)
- QEMU (Open Source)
- Virtual Server (MS)

## Hyperviseur BareMetal (sans OS sur l'Hôte)

- Xen (Open Source)
- VMWare ESX (VMWare)
- Hyper-V (MS)
- Linux KVM (Open Source)

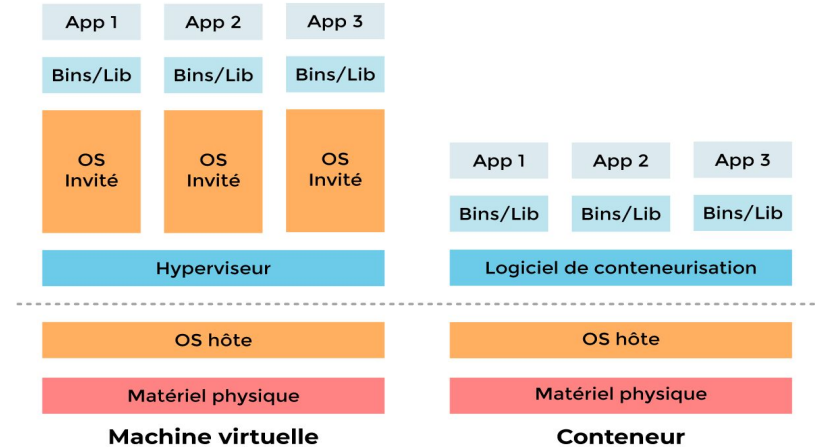


## Hébergeurs de VMs : Exemples Cloud

- AWS (Amazon)
- Azure (MS)
- GCP
- OVH
- Digital Ocean

# Le Conteneur

- Nouvelle solution de virtualisation basée sur des conteneurs
- L'Hôte qui accueille un conteneurs est constituée de :
  - Un OS
  - Des librairies
  - Des commandes
  - Un Container Engine
- Par rapport aux VM il manque l'OS
- Comment un conteneur fonctionne t-il s'il n'y a pas d'OS ? :
  - Utilisation de l'OS de la machine Hôte
  - Le conteneur accède à la machine Hôte grâce à l'Isolation :
    - Un conteneur est simplement un processus de la machine Hôte
    - Le conteneur s'appuie donc sur la machine Hôte de manière cloisonnée, isolée des autres processus de la machine Hôte sur lesquels il n'a aucune vision et aucun accès





## Avantages d'un conteneur

- Consommation réduite de CPU, RAM et de disque
- Plus de conteneurs
- Réduction du coût matériel
- Déploiement et démarrage très rapides

---

# Discussions et exemples pratiques pour illustrer les concepts



## Images et Conteneurs

- Docker repose sur deux concepts essentiels : les images et les conteneurs
- Une image est une sorte de modèle pour un conteneur
- Un conteneur est créé à partir d'une image
- Pour faire une analogie avec la programmation objet :
  - Une image est une classe.
  - Un conteneur est une extension de cette classe, de cette image.



# Layers

- Les images sont constituées de calques ou de couches (des Layers)
- Chaque calques apporte des modifications, ajout, suppression, changements à l'image pour former une nouvelle image
- L'empilement de tous ces calques forment l'image finale
- Tout comme on peut acheter des ingrédients pour faire une pizza au lieu de tout produire soi-même, on peut utiliser des images Docker existantes comme base et les modifier selon nos besoins, plutôt que de reconstruire l'ensemble à partir de zéro.
- Cela permet d'économiser du temps et de l'énergie tout en obtenant rapidement un produit final conforme à nos besoins.



# Registres

- Il s'agit du registre officiel administré par Docker, regroupant plus d'un million d'images publiques.
- On y trouve des images officielles et des images de la communauté
- Il est recommandé de choisir des images officielles en premier lieu, sinon celles qui ont une forte communauté d'utilisateurs, avec un grand nombre de téléchargements et de bonnes appréciations.
- Il est possible d'avoir un registre privé sur Docker Hub, accessible uniquement par vous et votre équipe, pour stocker des images sensibles.
- Le Docker hub est comparé à un GitHub ou GitLab pour les images de conteneurs, et il est recommandé, voire souhaitable, d'avoir son propre registre privé pour certaines images sensibles.



# FYI

- Play with docker :
  - Docker in Docker (DinD)
  - Proof Of Concept
  - <https://labs.play-with-docker.com/>
  - Session de 4H maxi
- hub.docker.com :
  - Analyser/Décrire les stats et les infos
  - Différencier : Image publique (Officielles) et image d'un user
- Digital Ocean :
  - <https://tuto.video/do> (200\$ pour 2 mois d'utilisation gratuite)
  - <https://docs.docker.com/engine/install/ubuntu/> (pour installer Docker CE)
- Tuto : <https://gitlab.com/devsecopscourses>



# Installation et Configuration de Docker

Guide pas à pas sur l'installation et la configuration de Docker, démonstration si possible



## Téléchargement et installation de Docker, suivi de Docker Compose

- Installation de Docker et Docker Compose sur un serveur via Digital Ocean, parmi les hébergeurs les plus abordables
- Possibilité d'installation sur une machine Linux locale, mais nécessitant des droits administratifs
- Précautions à prendre lors de l'installation chez certains hébergeurs Linux, tel que OVH sur des serveurs mutualisés
- Recommandation de suivre les instructions officielles de Docker pour l'installation, avec des options adaptées à différentes distributions Linux
- Vérification de l'installation réussie via la commande "docker-compose version", confirmant l'installation de Docker et Docker Compose
- <https://docs.docker.com> (pour installer Docker CE)



# Utilisation de Docker pour le Développement et le Déploiement



## Premiers conteneurs, premières commandes

Commandes docker : qui permet de donner des ordres au Docker Engine qui pilote tout notre environnement Docker

- ***docker run*** : qui permet de créer un conteneur en téléchargeant éventuellement l'image si elle n'est pas déjà présente dans le cache local. Le téléchargement se faisant depuis le registre Docker Hub
  - ***-t*** : Pseudo-TTY = Terminal (STDIN+STDOUT)
  - ***-i*** : Interactif (au clavier via le Pseudo-TTY)
  - Utilisé conjointement ***-ti*** ressemble à une connexion en SSH
- ***docker images*** : qui permet de lister des images présentes localement sur notre machine dans le cache de docker
- ***docker ps*** : qui liste les conteneurs créés sur notre machine



## Les IDs et les raccourcis

- Les **IDs** sont des identifiants uniques, des séries de chiffres hexadécimaux
  - **IDs** : nécessaires pour **manipuler** les **objets Docker** (Images, Conteneurs, etc)
- Exemple de commande "***docker history***" nécessitant l'**ID** d'une image pour afficher ses couches
- Astuce pour accélérer la saisie des **IDs** en ne spécifiant que le début tant qu'il n'y a pas de conflit
- **Utilisation de la complétion** dans le shell, similaire à celle dans Bash, pour faciliter la saisie des commandes Docker



## Préserver les données

- Relancer un conteneur (***docker start*** et ***stop***)
- Conserver un conteneur, même après son arrêt : Introduction sur la nécessité de pouvoir arrêter et redémarrer un conteneur sans perdre ses données
  - Règle d'or : les modifications dans un conteneur sont TEMPORAIRES
- Exemple d'utilisation de la commande "***docker run***" avec l'option "***-rm***" pour créer un conteneur et constatation de la perte de données à l'arrêt du conteneur
  - ***docker run*** : crée toujours un nouveau conteneur
  - Identification de l'option "***-rm***" comme la cause de la suppression des données à l'arrêt du conteneur

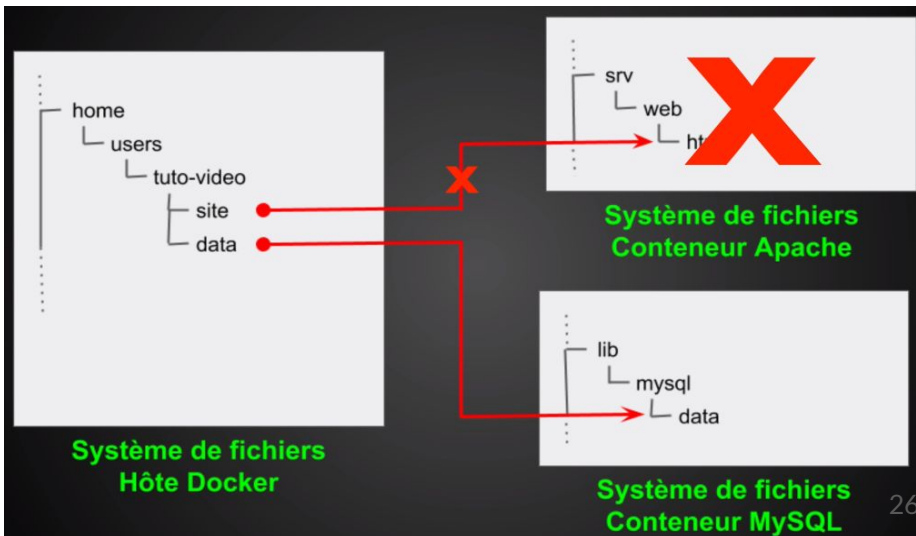
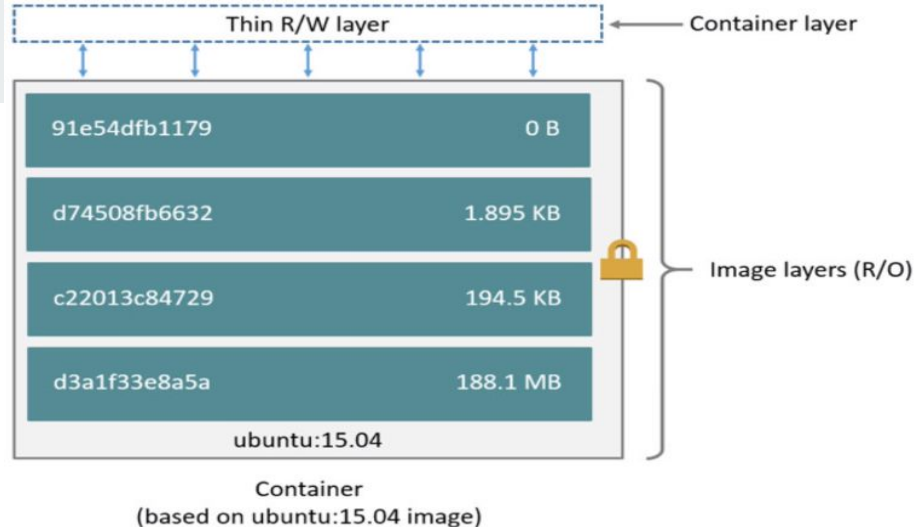


## Préserver les données

- Suppression de l'option "**-rm**" et constatation persistante de la perte de données à l'arrêt du conteneur.
- Utilisation de la commande "**docker ps --help**" pour obtenir de l'aide sur les options de la commande "**docker ps**" et découverte de l'option "**-a**" pour afficher tous les conteneurs, même ceux arrêtés.
- Utilisation de l'option "**-a**" pour confirmer la suppression du conteneur à l'arrêt et résolution du problème avec l'utilisation de la commande "**docker start**" pour redémarrer un conteneur existant.
- Conclusion sur la préservation des données dans Docker, l'utilisation des commandes "**docker start**" et "**docker stop**", et l'importance de l'expérimentation personnelle pour mieux comprendre Docker

# Les volumes

- Un volume permet de **monter un fichier ou un répertoire** de la machine hôte directement dans le conteneur
- Protéger le montage avec le mode “Lecture seule” (*ro*)
- Tester plusieurs versions d'un environnement





## Le réseau, partie 1

- Mapper des ports TCP/IP de l'Hôte vers le conteneur
- Trouver les **ports utilisés dans le conteneur**
- ***docker inspect*** (sur images, conteneurs, etc.)
- **Logs** affichés par le cours d'exécution



## Détacher, attacher

- Lancer le conteneur en **mode Détaché**
- Se **rattacher** à un conteneur détaché
- Se **détacher** d'un conteneur
- Accéder aux **logs** d'un conteneur détaché (**docker logs**)



## Création d'image

- **Améliorer** une image existante
- Préparer un fichier **Dockerfile**
- Construire une image avec ***docker build***
- Exécuter une commande dans un conteneur avec ***docker exec***



## Le réseau, partie 2

- Pilotes de réseau natifs :
  - **none, bridge, host, overlay, Macvlan**
- Cloner un pilote bridge
- **docker network create**
- **docker network ls**
- **docker network rm**
- Connecter et déconnecter un bridge cloné sur un conteneur actif



## Les volumes, partie 2

- Similitudes et différences entre :
  - volumes mappés et volumes managés
- Créer un volume managé : *docker volume create*
- Monter un volume managé : *docker volume -v*
- Lister les volumes managés : *docker volume ls*
- Inspecter un volume managé : *docker volume inspect*
- Transférer des données du conteneur vers l'Hôte



# Docker Compose

- Docker compose le *chef de la cuisine Docker*
- *docker-compose.yml*, la recette de cuisine Docker
- Comment créer *des services, des réseaux et des volumes*
- Comment lancer, arrêter, supprimer un environnement avec **docker-compose up, down, start et stop**



- Qu'est ce que **Portainer** ?
  - interface Web
  - permet de piloter graphiquement les conteneurs d'un ou plusieurs hôtes
  - ne retire pas la nécessité d'utiliser la ligne de commande
  - pratique pour certaines tâches d'administration
- Comment installer Portainer
- Le **Dashboard** (Images, Conteneurs, Volumes, Réseaux)
- Comment créer un **conteneur avec Portainer**

---

# Evaluation du cours